

System Analysis & Advise Document

JabberPoint Slideshow Application

Software Quality
Nathan P. Pais D'costa - 5045800 Stefani Margaritova - 4967399
02/06/2026
Version 0.1

Table of Contents

Table of Contents.....	2
1. Introduction & Scope.....	3
1.1 Purpose of this Document	3
1.2 Scope	3
1.3 Document Map	3
2. Current System Overview	3
2.1 Functional Summary.....	3
2.2 Architectural Layers.....	4
2.3 Component Responsibilities	4
3. As-Built Class Diagram	6
3.1 Diagram Walkthrough	7
4. SOLID Violations in the Current Design	8
4.1 Single Responsibility Principle (SRP)	8
4.2 Open-Closed Principle (OCP).....	8
4.3 Liskov Substitution Principle (LSP)	9
4.4 Interface Segregation Principle (ISP).....	9
4.5 Dependency Inversion Principle (DIP)	9
5. Other Design Observations	10
7. Updated (To-Be) Class Diagram	14
7.1 What Changed and Why.....	Error! Bookmark not defined.
8. Design Principle Compliance Summary	Error! Bookmark not defined.
9. Risks and Trade-offs of the Redesign	Error! Bookmark not defined.
10. Recommendations for Development Process.....	Error! Bookmark not defined.
10.1 Development Street (DTAP)	Error! Bookmark not defined.
10.2 Automated Testing & Code Quality.....	Error! Bookmark not defined.
10.3 Build & Executable Location	Error! Bookmark not defined.
Appendix A - Original (As-Built) Class Diagram.....	15
Appendix B - Refactored (To-Be) Class Diagram	16
Appendix C - Requirements Traceability Table	Error! Bookmark not defined.

PART A — SYSTEM ANALYSIS

1. Introduction & Scope

1.1 Purpose of this Document

This document analyzes the current design of the JabberPoint slideshow application and evaluates it against the SOLID design principles. Part A (System Analysis) describes the system as it currently exists and identifies concrete design weaknesses. Part B (Advise) proposes specific changes to address those weaknesses, presents the resulting redesign, and advises on the development process used to deliver and maintain the software going forward.

1.2 Scope

This analysis covers all classes contained in the project's src folder, specifically:

- The presentation domain model: Presentation, Slide, Slideltem, TextItem, BitmapItem, Style
- The data access layer: Accessor, XMLAccessor, DemoPresentation
- The Swing/AWT user interface layer: SlideViewerFrame, SlideViewerComponent, MenuController, KeyController, AboutBox
- The application entry point: JabberPoint

Out of scope for this analysis: the build tooling, the DTAP pipeline, and automated testing/code quality setup, which are addressed separately in Section 10 (Recommendations for Development Process) once the redesign in Part B is finalized.

1.3 Document Map

Part A (System Analysis) describes the current system. Part B (Advise) proposes and justifies changes. Appendices contain full diagrams and traceability tables.

2. Current System Overview

2.1 Functional Summary

JabberPoint is a simple desktop slideshow viewer written in Java using AWT/Swing. From a user's perspective, the application:

- Opens either a built-in demo presentation or a presentation loaded from an XML file
- Displays one slide at a time, where each slide consists of a title and an ordered list of items (either indented text lines or bitmap images)
- Allows navigation between slides using the keyboard (next/previous) or the View menu
- Allows the user to open a new XML presentation file, save the current presentation to XML, or start a new (empty) presentation via the File menu
- Provides a "Go to" function to jump directly to a slide by number
- Shows an About dialog with information about the application

2.2 Architectural Layers

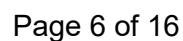
The current code can be grouped into four informal layers. There is no package structure in place, all classes sit in a single default package but the following grouping reflects each class's actual role:

1. Domain model: Presentation, Slide, SlidelItem (abstract), TextItem, BitmapItem, Style. These classes represent the data of a presentation and know nothing about how it is displayed, loaded, or saved.
2. Data Access: Accessor (abstract), XMLAccessor, DemoPresentation. These classes are responsible for populating a Presentation object from some source (an XML file or hardcoded demo content) and, in the case of XMLAccessor, writing it back out.
3. User interface: SlideViewerFrame, SlideViewerComponent, MenuController, KeyController, AboutBox. These classes build the AWT/Swing window, render the current slide, and translate user input (menu clicks, key presses) into actions on the Presentation.
4. Entry point: JabberPoint. Wires the above together and starts the application.

2.3 Component Responsibilities

Class	Current Responsibility
JabberPoint	Application entry point. Parses the command-line argument (an XML filename, if given), creates the Presentation, chooses between XMLAccessor and the demo Accessor, and creates the SlideViewerFrame.
Presentation	Holds the list of Slide objects for the current presentation, the presentation's title, and the index of the currently displayed slide. Provides navigation (next/previous slide, jump to slide number) and notifies the view component to redraw when the slide changes.
Slide	Holds a single slide's title and an ordered Vector of SlidelItem objects. Computes the scale factor for the slide area and draws all of its items by delegating to each SlidelItem's draw method.
SlidelItem / TextItem / BitmapItem	SlidelItem is an abstract base class defining a shared "level" (indentation depth) and abstract draw/getBoundingBox operations. TextItem represents one line of indented text; BitmapItem represents an image loaded from disk. Each subclass implements its own drawing and sizing logic.
Style	Defines the indentation, font, font size, line spacing, and color used to render a given outline level. Holds a static array of pre-built Style objects, one per level.

Class	Current Responsibility
Accessor / XMLAccessor / DemoPresentation	Accessor is an abstract class declaring loadFile/saveFile operations and a static factory method for the demo accessor. XMLAccessor implements loading and saving a presentation to/from an XML file using a DOM parser. DemoPresentation implements loadFile by programmatically constructing a fixed, hardcoded demo presentation, and does not support saving.
SlideViewerFrame / SlideViewerComponent	SlideViewerFrame builds the top-level application window attaches the menu bar and key listener, and sizes/positions the window. SlideViewerComponent is the drawing surface that renders the current slide's title and items and listens for presentation changes to trigger a repaint.
MenuController / KeyController	MenuController builds the File/View/Help menus, wires each menu item to an action (open, save, new, exit, next, previous, go to, about), and constructs an XMLAccessor when Open or Save is chosen. KeyController listens for key presses (e.g. PgDn, PgUp, Enter, arrows, "q") and translates them into Presentation navigation calls or application exit.
AboutBox	Displays a static informational dialog box describing the application, triggered from the Help menu.



3.1 Diagram Walkthrough

The class diagram shows two parallel inheritance hierarchies and one central composition structure.

The first hierarchy is the slide-item model: the abstract class `SlideItem` defines a shared "level" attribute and two abstract operations, `getBoundingBox` and `draw`. `TextItem` and `BitmapItem` each extend `SlideItem` and provide their own implementation of these two operations, `TextItem` renders a line of styled text using `AttributedString/TextLayout`, while `BitmapItem` renders an image loaded via the Toolkit. Both subclasses depend on `Style` to determine how their content should look at the current outline level.

The second hierarchy is the data access model: the abstract class `Accessor` declares `loadFile` and `saveFile`, and exposes a static factory method, `getDemoAccessor()`, used to obtain a built-in demo data source. `XMLAccessor` and `DemoPresentation` both extend `Accessor`. `XMLAccessor` implements both methods by reading/writing an XML file with a DOM parser, instantiating `TextItem` and `BitmapItem` objects as it parses each 'item' element. `DemoPresentation` implements `loadFile` by hardcoding a fixed set of `Slide` and `SlideItem` objects directly in Java code, and implements `saveFile` by unconditionally throwing an exception, since a hardcoded demo cannot meaningfully be "saved."

The composition structure sits at the center of the model: a `Presentation` holds an `ArrayList` of `Slide` objects, and each `Slide` holds a `Vector` of `SlideItem` objects. This means a `SlideItem` cannot outlive the `Slide` that contains it, and a `Slide` cannot outlive the `Presentation` that contains it. A strict ownership (whole/part) relationship, shown with filled diamonds on the diagram.

The user-interface classes sit outside this core model and depend on it rather than being part of it. `SlideViewerFrame` extends `JFrame` and, in its constructor, creates a `SlideViewerComponent`, a `KeyController`, and a `MenuController`, wiring them to the same `Presentation` instance. `SlideViewerComponent` extends `JComponent` and observes the `Presentation`, repainting itself whenever the current slide changes. `KeyController` extends `KeyAdapter` and translates key events directly into calls on `Presentation` (`nextSlide`, `prevSlide`, `exit`). `MenuController` extends `MenuBar` and, beyond building the menu structure, directly constructs an `XMLAccessor` inside its own action-listener code whenever the user chooses Open or Save. This direct dependency on a concrete class, rather than on the `Accessor` abstraction, is one of several issues examined in Section 4.

Finally, `JabberPoint` is the entry point: it creates the `Presentation`, decides (based on whether a filename was supplied) whether to use `XMLAccessor` or `Accessor.getDemoAccessor()` to load it, and creates the `SlideViewerFrame` that displays it.

4. SOLID Violations in the Current Design

4.1 Single Responsibility Principle (SRP)

The Single Responsibility Principle states that a class should have only one reason to change.

MenuController violates this principle. In addition to its stated responsibility of building the menu structure, its constructor also directly performs file-loading and file-saving orchestration: the action listeners for "Open" and "Save" each construct a 'new XMLAccessor()' and call loadFile/saveFile on it directly.

This means MenuController has at least three independent reasons to change:

1. The menu structure or labels change.
2. The file format or data access mechanism changes.
3. The error-dialog/messaging behavior changes.

A change to how presentations is continued, for example, adding a JSON format would require modifying a class whose stated purpose is building a menu, not performing I/O.

4.2 Open-Closed Principle (OCP)

The Open-Closed Principle states that a class should be open for extension but closed for modification, it should be possible to add new behavior without editing existing, working code.

XMLAccessor violates this principle in two places. First, in loadSlideItem, the type of each slide item is determined with a chain of string comparisons:

```
if (TEXT.equals(type))
{
    slide.append(new TextItem(level, item.getTextContent()));
}
else
{
    if (IMAGE.equals(type))
    {
        slide.append(new BitmapItem(level, item.getTextContent()));
    }
    else
    {
        System.err.println(UNKNOWN_TYPE);
    }
}
```

Second, in saveFile, the same problem appears in reverse: each SlideItem is checked with 'instanceof TextItem' and 'instanceof BitmapItem' to decide how to serialize it. In both cases, adding a new kind of SlideItem, for example, a shape or a hyperlink item, requires opening and modifying XMLAccessor itself, rather than simply adding a new class that the existing loading/saving code can already accommodate.

4.3 Liskov Substitution Principle (LSP)

The Liskov Substitution Principle states that an instance of a subtype should be substitutable for an instance of its supertype without altering the correctness of the program. That is, a subclass must honor the behavioral contract implied by its parent.

DemoPresentation violates this principle. Accessor's contract implies that any concrete Accessor supports both loadFile and saveFile where both are declared as abstract, unconditional operations. However, 'DemoPresentation.saveFile()' does not save anything; it unconditionally throws 'new IllegalStateException("Save As->Demo! called")'. Any code that holds a reference typed as Accessor and calls saveFile will work correctly for an XMLAccessor but crash for a DemoPresentation. The subtype cannot be safely substituted for its supertype, which is an LSP violation.

4.4 Interface Segregation Principle (ISP)

In the current design, Accessor only declares two methods; loadFile and saveFile, so on its own it is small enough that ISP is not yet visibly violated. However, it is closely related to the LSP violation above: the reason DemoPresentation is forced to implement a saveFile method it cannot honor is that Accessor bundles "loading" and "saving" into a single abstraction that every implementer must support in full, even when a given implementer only meaningfully supports one of the two.

If Accessor were to grow further, this bundling would become a more visible ISP problem, since implementers like DemoPresentation would be forced to depend on an increasing number of methods they cannot support.

4.5 Dependency Inversion Principle (DIP)

The Dependency Inversion Principle states that high-level modules should not depend on low-level modules directly, however, both should depend on abstractions.

Although the Accessor abstraction is already provided, callers do not need to know which concrete data access mechanism is in use, MenuController's Open and Save action listeners each call 'Accessor xmlAccessor = new XMLAccessor()' directly, rather than receiving an Accessor from an external class. This means MenuController cannot be reused or tested with a different Accessor implementation such as DemoPresentation, a mock for testing, or a future JSON-based accessor without editing MenuController's source code directly.

5. Other Design Observations

Beyond the five SOLID principles, a few additional design issues are worth noting, as they affect maintainability:

- No package structure: all classes live in a single default package, which makes the architectural layers described in Section 2.2 implicit rather than enforced by the code itself.
- Magic strings and hardcoded file names: MenuController hardcodes the filenames "test.xml" and "dump.xml" as constants (TESTFILE, SAVEFILE) used for Open and Save, meaning the application always loads/saves the same two files regardless of user intent .
- Minimal/inconsistent error handling: XMLAccessor.loadFile catches parsing-related exceptions and only logs them to System.err, silently continuing rather than informing the user or failing safely. DemoPresentation.saveFile(), by contrast, throws an unchecked exception with no caller-side handling visible in MenuController for that specific case beyond the generic IOException catch, which it would not actually catch.
- Tight coupling between UI and domain logic: SlideViewerComponent, MenuController, and KeyController all hold direct references to Presentation and call its methods directly with no intervening abstraction such as an observer interface or command pattern, which is workable at this scale but would make the UI harder to unit test or swap out independently of the domain model.

PART B — ADVISE DOCUMENT

6. Proposed Design Changes

6.1 Design Pattern Overview

#	Pattern / Fix	Primary Target Classes	Resolves
1	Observer	Presentation, SlideViewerComponent	Decouples UI from domain model (supports Section 5 observation; foundation for the other fixes)
2	Factory	XMLAccessor.loadSlideItem	OCP
3	Strategy	SlideItem subtypes, XMLAccessor.saveFile	OCP
4	Command	MenuController	SRP, DIP
5	Interface split (ISP)	Accessor to PresentationLoader / PresentationSaver	ISP, LSP

6.2 Observer Pattern — Decoupling the UI from the Domain Model

Presentation becomes the subject in an Observer relationship. It holds a list of registered PresentationObserver instances and notifies each one through an update() call whenever setSlideNumber changes the current slide. SlideViewerComponent implements PresentationObserver and registers itself with Presentation at construction time, in SlideViewerFrame. This replaces the previous arrangement, where SlideViewerComponent held a direct reference to Presentation and had no defined mechanism for being told when the model changed.

This change does not resolve a named SOLID violation by itself. Instead, it directly addresses the tight UI-to-domain coupling flagged in Section 5, and it gives the rest of the redesign a cleaner foundation: once the view is decoupled from the model through a defined notification contract, the model can be modified by other means, including the Command objects introduced in Section 6.5, without the UI needing to know how or why a change occurred.

6.3 Factory Pattern — Fixing OCP in Slideltem Loading

The if/else string-comparison chain previously in XMLAccessor.loadSlideltem is replaced by a SlideltemFactory. XMLAccessor now delegates to a SlideltemFactory field, calling createSlideltem(level, type, content), which returns the appropriate TextItem or BitmapItem. The TEXT, IMAGE, and UNKNOWN_TYPE constants that previously lived in XMLAccessor moved into SlideltemFactory along with the comparison logic itself.

This directly resolves the loading-side OCP violation identified in Section 4.2. Adding a new Slideltem subtype, for example a hyperlink item, now means extending SlideltemFactory's creation logic rather than reopening and editing XMLAccessor.loadSlideltem. XMLAccessor itself no longer needs to change when the set of supported item types changes.

6.4 Strategy Pattern — Fixing OCP in Slideltem Saving

The instanceof TextItem / instanceof BitmapItem chain previously in XMLAccessor.saveFile is replaced by two new abstract methods on Slideltem: getXMLKind(), which returns the XML "kind" attribute value for that item type, and getXMLContent(), which returns the text to write inside the XML element. TextItem and BitmapItem each implement both methods according to their own representation, returning "text"/the text content and "image"/the image filename respectively. XMLAccessor.saveFile now calls slideltem.getXMLKind() and slideltem.getXMLContent() polymorphically, with no type-checking of any kind.

This resolves the other half of the Section 4.2 OCP violation, the saving side. This is a Strategy rather than a Factory because it varies behavior, namely how an existing object serializes itself, rather than creating an object. A new Slideltem subtype now only needs to implement these two methods to be saved correctly; XMLAccessor.saveFile does not need to know that subtype exists.

6.5 Command Pattern — Fixing SRP and DIP in MenuController

Each menu action (Open, Save, New, Exit, Next, Prev, GoTo, About) is now its own class implementing the Command interface, with a single execute() method: OpenCommand, SaveCommand, NewCommand, ExitCommand, NextSlideCommand, PrevSlideCommand, GotoSlideCommand, and AboutCommand. MenuController's constructor builds these eight Command instances once, then wires each menu item's ActionListener to call execute() on the corresponding command.

The file-loading and file-saving logic that used to live inline inside MenuController's action listeners now lives entirely inside OpenCommand and SaveCommand. Critically, OpenCommand and SaveCommand are constructed with a PresentationLoader and a PresentationSaver respectively, not a concrete XMLAccessor. Internally they hold only that interface reference and call loadFile/saveFile through it. MenuController still constructs a single XMLAccessor instance, since something has to create the concrete object that gets handed to these commands, but this now happens once, at one composition point in the constructor, rather than being repeated inside each action listener as in the original design.

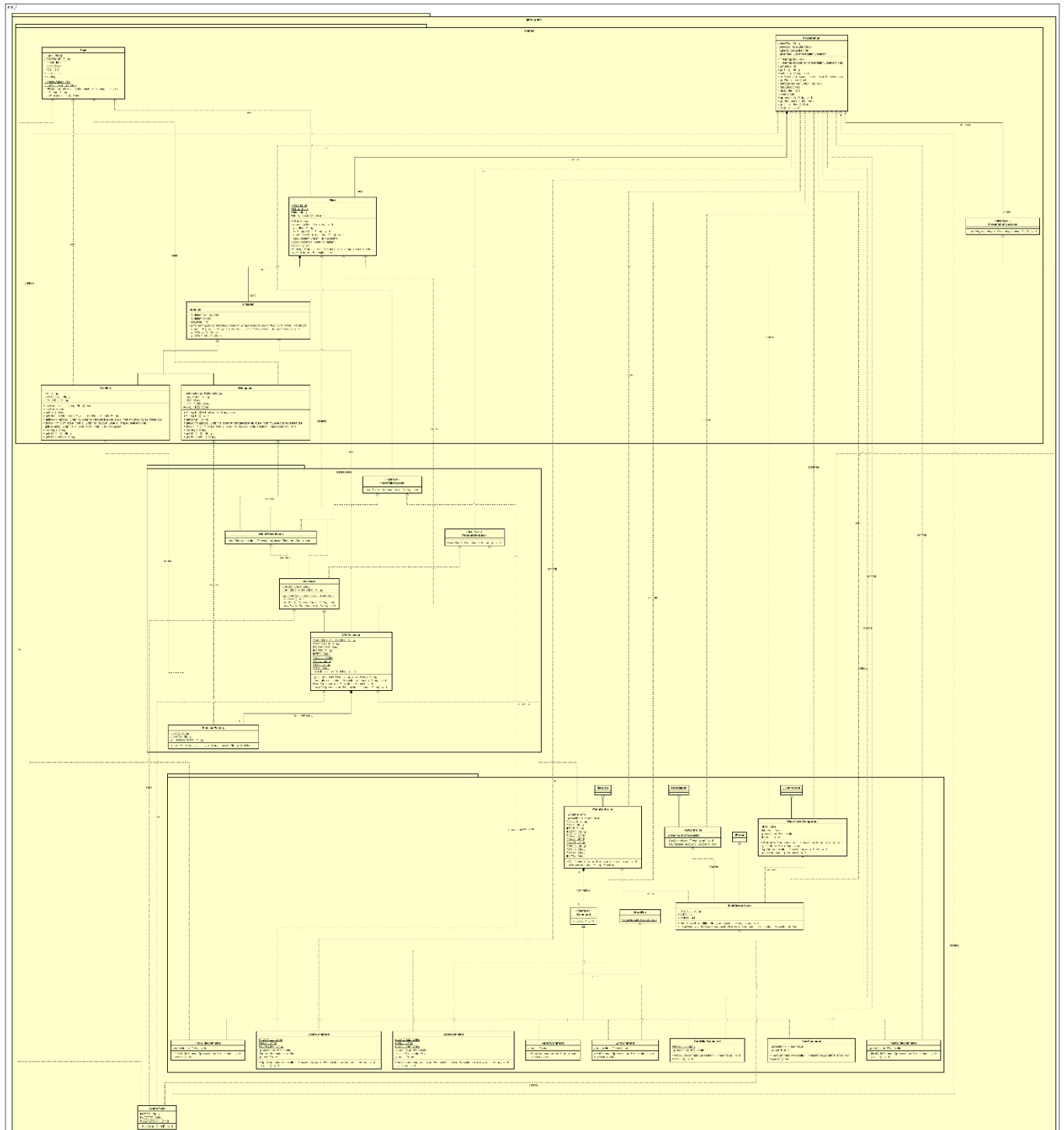
This resolves the SRP violation identified in Section 4.1: MenuController's action listeners no longer contain file I/O logic, parsing, or error-dialog handling, that responsibility has moved into the individual Command classes. It also resolves the DIP violation identified in Section 4.5: OpenCommand and SaveCommand depend on the PresentationLoader/PresentationSaver abstractions, not on XMLAccessor directly, so either command could be constructed with a different implementation, for example DemoPresentation or a test mock, without modifying OpenCommand, SaveCommand, or MenuController's menu-building logic.

6.6 Interface Split — Fixing ISP and LSP in Accessor

Accessor's two methods, loadFile and saveFile, are split into two single-method interfaces: PresentationLoader (loadFile) and PresentationSaver (saveFile). Accessor becomes an abstract class that implements both interfaces, preserving its DEMO_NAME and DEFAULT_EXTENSION constants and its getDemoAccessor() factory method, now declared to return PresentationLoader rather than Accessor. XMLAccessor still extends Accessor and so still implements both interfaces. DemoPresentation, however, now implements only PresentationLoader directly, since it never meaningfully supported saving.

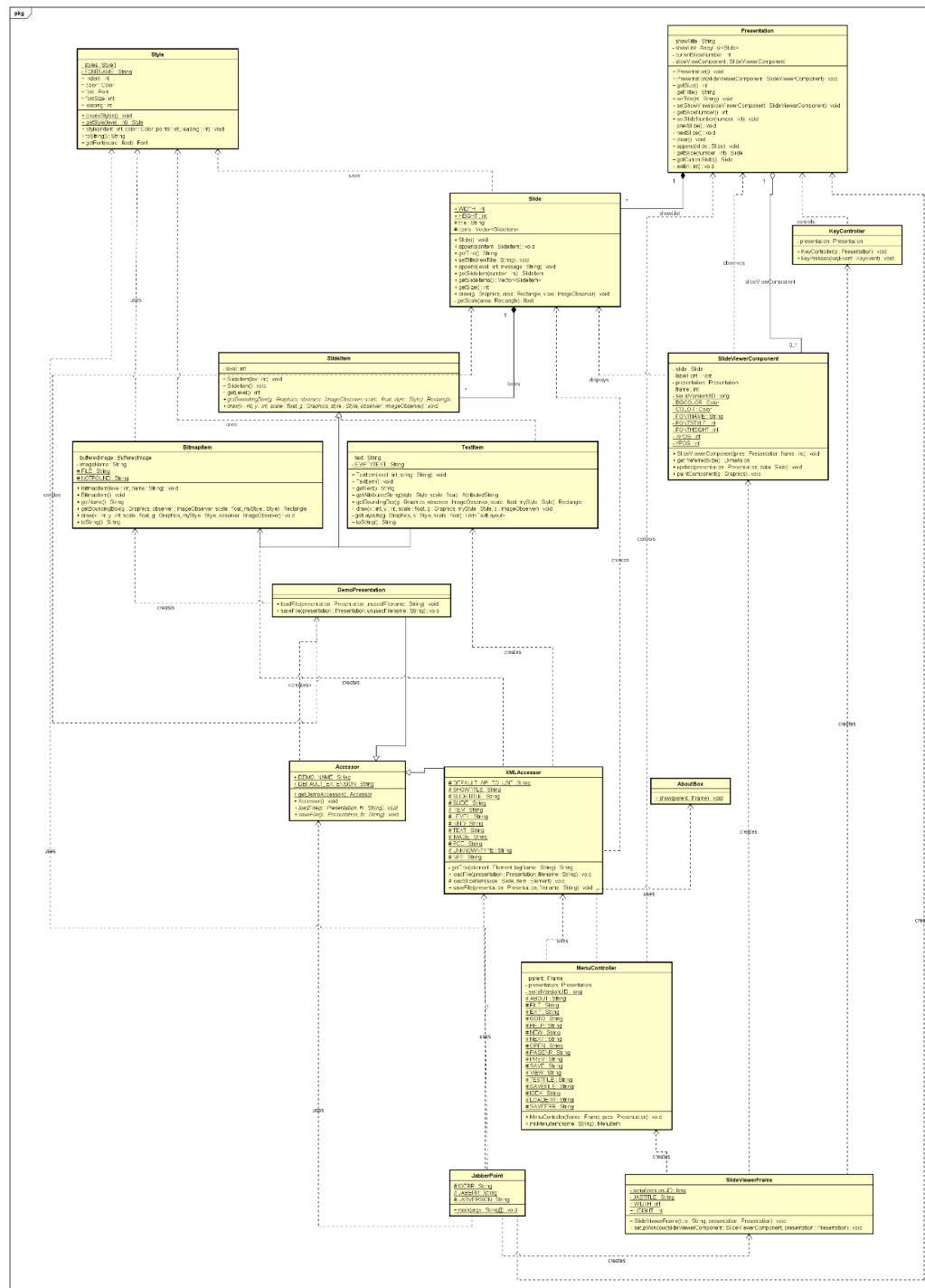
This is an ISP fix, addressing Section 4.4: no implementer is now forced to depend on a method it doesn't use. It also resolves the Section 4.3 LSP violation as a direct side effect: DemoPresentation no longer declares a saveFile method at all, so the broken implementation that unconditionally threw IllegalStateException("Save As->Demo! called") has been deleted entirely, not stubbed or suppressed. Calling saveFile on a DemoPresentation reference is now a compile-time error rather than a runtime exception, which is a stronger guarantee than try/catch could provide.

7. Updated Class Diagram



APPENDICES

Appendix A - Original (As-Built) Class Diagram



Appendix B - Refactored Class Diagram

